10.10.2024

Ersetzungssysteme

1. Menge möglicher Kontexte, in denen Ersetzungen definiert sind
   => „Wo Ort"
2. Ersetzungsregeln mit input ⟶ output
   => Liste
3. Metaregeln: Regelt die Anwendung von Ersetzungsregeln

Produktionen / Ersetzungsregeln

Textersetzungsregeln, z.B. x -> y

Text: mxp -> myp

Ein Textersetzungssystem (Semi-Thue-System) $ST = (V, P)$ ist durch ein Alphabet V und die Regelmenge P festgelegt

Markov-Algorithmen

Semi-Thue-System mit folgenden Besonderheiten:

1. Linke Regelseite darf nicht ε (Leer) sein
2. Stopregel: Implizitert Ende der Ausführung (Rechte Seite beginnt mit ".")
3. Erste anwendbare Regel anwenden
   -> Wenn nicht möglich: Ende der Abarbeitung

Bsp.: Strich -> Dualzahlen:

P: { 1. @I -> I@ Hier @ als „Schiffchen" (Platzhalter)
2. @ -> 1
3. @ -> 0
4. I -> @I }

$I^4 = IIII$: (4) => @IIII
(1) => I@III
(1) => II@II
(1) => III@I
(3) => III0
(4) => @III0

(1) => I@10
(3) => I100
(4) => @I100
(2) => 1100

Begriffe

- Alphabet: Endliche Menge der (Eingabe-) Symbole
- Symbol: Schriftzeichen oder Zusammensetzungen, zBs. „-.-"
- Wort: Zeichenkette bestehend aus Zeichen bzw. Symbolen des Alphabets
  ↳ Leeres Wort: ε ; $|\varepsilon| = 0$
- Wortmenge: Menge aller möglichen Wörter (abzählbare unendliche Menge)
- Sprache: Teilmenge der Wortmenge

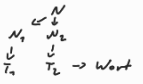
Kontextfreie Grammatik

$G = (N, T, S, P)$ ist ein $ST = (V, P)$ mit:

- Alphabet V ist aufgeteilt in:
  1. N: Nichtterminale Symbole
     inkl. Startsymbol S
     => Alle Ersetzbaren Symbole
  2. T: Terminale Symbole
     => finale Symbole
- Jede Linke Seite aus P hat genau ein $x \in N$

G definiert eine Sprache

$L(G) = \{w \mid S \Rightarrow^* w$, w ist Wort über T$\}$
$= \{w \mid$ w ist terminale Blattfolge im Syntaxbaum$\}$



Backus-Naur-Form (BNF & EBNF)

P { Satz -> Subjekt Prädikat Objekt,
Prädikat -> sieht
Subjekt -> james
Subjekt -> lukas
...

BNF:

<satz> ::= <subjekt> <prädikat> <objekt>
<prädikat> ::= "sieht" | "geht" | ...
<subjekt> ::= "james" | "lukas" | ...

<Nichtterminale>
"Terminale"

BNF:

<expression> ::= <expression> + <term> | <term>
<term> ::= <term> "*" <factor> | <factor>
<factor> ::= "(" <expression> ")" | "zahl"

EBNF (Erweiterte BNF)

() Gruppierungen
[] Optionaler Anteil
{} Wiederholbarer Anteil
*, + Wiederholung
*: beliebig oft, +: mindestens einmal

Bsp: Arithmetische Ausdrücke

G_{Az}: E -> E+T | T
T -> T*F | F
F -> (E) | zahl

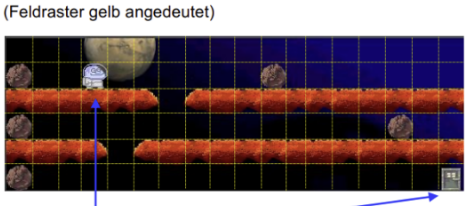
EBNF:

<expression> ::= <term> { "+" <term> }
<term> ::= <factor> ("*" <factor>)*

Beispiel

Beispiel: Graphische Ersetzungssysteme, hier: Stagecast

- Kontext:
  (Feldraster gelb angedeutet)



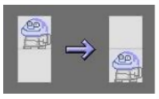
- Ziel: Der Astronaut soll die Tür erreichen

Regeln:

- Regel „gehe auf ein leeres Feld nach rechts":



- Regel „Schwerkraft wirkt beim Blick nach links":



- Regel „Am rechten Fels nach links umkehren":



Zusätzlich: Alle links-rechts spiegelbildlichen Regeln

Metaregeln:

- Zu jeder Figur gibt es eine Liste von Regeln.
- Davon wird immer die erste anwendbare gewählt.
  Es kommt also auf die Reihenfolge der Regeln an!
- Solange es geht, werden Regeln angewandt.
- Mehrere Figuren werden reihum abwechselnd bewegt (nur die gerade beweglichen)

- Was bewirken folgende Regellisten?

Regelliste A ✓	Regelliste B ✗
1. Schwerkraft wirkt (links/rechts)	1. Gehen (links/rechts)
2. Gehen (links/rechts)	2. Am Felsen umkehren (links/rechts)
3. Am Felsen umkehren (links/rechts)	3. Schwerkraft wirkt (links/rechts) ?!

=> Reihenfolge wichtig;
B: Regel 3 wird niemals angewendet

Verwendung von Metaregeln:

- Bsp.:
  - immer die erste passende Regel anwenden
  - alle anwendbaren Regeln gleichzeitig anwenden
    ↳ Überschneidung beachten
  - zyklische Anwendung
  - Wiederholungen
    - einmal
    - n mal
    - so oft es geht
  - Auf Regel X folgt Regel y
- Festlegen einer Default-Metaregel
  (sofern nicht anders festgelegt):
  Eine anwendbare zufällig gewählte Regel

Rekursion

-> Divide and conquer

Beispiel:

Spezifikation von ggT:

1. Eingabedaten: $(a,b) \in \mathbb{N}_0 \times \mathbb{N}_0 \setminus \{(0,0)\}$
2. Ergebnis: Größtes c, welches sowohl a als auch b teilt

   „z teilt x": $\exists x' \in \mathbb{N}_0 : x = x' \cdot t$
3. Operatoren: +, -, *, div, mod

   <, ≤, =, ≠, ≥, >

Funktionsdefinition:

$ggT(a,b) = ggT(a-b,b)$ falls $a > b > 0$

(ggT = formale Parameter; Bedingung ↳ Müssen den ganzen Definitionsbereich abdecken.)

Funktionsaufruf: $ggT(78,54)$ (aktuelle „lokale" Parameter)

Vervollständigung durch weitere Definitionszeilen:

$ggT(a,b) = ggT(a,b-a)$, falls $0 < a < b$

$ggT(a,a) = a$, falls $0 < a$

Alternative Bedingungen:

■ ggT1: Überlappende Bedingungen

$$ggT1(a,b) = \begin{cases} a, \text{ falls } a=b \text{ und } a>0 \\ a, \text{ falls } b = 0 \\ b, \text{ falls } a = 0 \\ ggT1(a-b,b), \text{ falls } a \geq b > 0 \\ ggT1(a,b-a), \text{ falls } b \geq a > 0 \end{cases}$$

```
ggT1(54,78) =
ggT1(54,24) =
ggT1(30,24) =
ggT1(6,24) =
ggT1(6,18) =
ggT1(6,12) =
ggT1(6,6) =
  6
  ggT1(0,6) = 6
  ggT1(6,0) = 6
```

ggT2: Effizientere Fassung

(Ersetze „a – b" durch „a *mod* b" etc.)

$$ggT2(a,b) = \begin{cases} b, \text{ falls } a = 0 \\ a, \text{ falls } b = 0 \\ ggT2(a \bmod b, b), \text{ falls } a \geq b > 0 \\ ggT2(a, b \bmod a), \text{ falls } b \geq a > 0 \end{cases}$$

```
ggT2(54,78) = ggT2(54,24) = ggT2(6,24) = ggT2(6,0) = 6
```

Rekursiv vs. Iterativ

Begriffe

Algorithmen

- Algorithmen beschreiben,
  - welche Operationen
  - in welcher Reihenfolge anzuwenden sind,

  um ein Problem zu lösen.
- Bei Markov-Algorithmen:
  - Regeln legen die Operationen fest.
  - Metaregeln geben die Reihenfolge vor.
- Bei Rekursionsgleichungen:
  - Erlaubte Operationen sind in der Spezifikation vorgegeben.
  - Die Reihenfolge ergibt sich aus dem Aufbau der Gleichungen, den Anwendbarkeitsbedingungen und der Default-Metaregel.

Begriffsbildung: deterministisch vs. determiniert

- Rekursionsgleichungen mit überlappenden Bedingungen (wie ggT1) legen die möglichen Ersetzungen nicht eindeutig fest (Default-Metaregel auf Folie 11).

  Dies bezeichnet man als nichtdeterministisch.
- Demgegenüber sind Markov-Algorithmen aufgrund ihrer Metaregel (Besonderheit #3 auf Folie 16) stets deterministisch.
- Kontextfreie Grammatiken sind „hochgradig nichtdeterministisch".
- ggT1 zeigt, dass nichtdeterministische Algorithmen durchaus eindeutige Ergebnisse produzieren können, also determiniert sein können.

Begriffsbildung: Terminierung und Korrektheit

- Ein Algorithmus terminiert, wenn er
  - für alle laut Spezifikation zulässigen Eingaben
  - nach endlich vielen Schritten ein Ergebnis produziert.
- Ein Algorithmus ist partiell korrekt, wenn er bezüglich der Spezifikation „zutreffende" Ergebnisse produziert:
  - Korrektheitsuntersuchungen setzen eine präzise Spezifikation voraus.
  - Typische Techniken:
    - Beweise (z.B. mittels Induktion)
    - Testen (kann allerdings höchstens Fehler finden)
- Ein Algorithmus heißt korrekt, wenn er terminiert und partiell korrekt ist.

Begriffsbildung: Effektivität vs. Effizienz

- Ein Algorithmus ist effektiv, wenn er das Gewünschte leistet.
- Ein Algorithmus ist effizient, wenn er mit den Ressourcen – insbesondere Rechenzeit und Speicherplatz – sparsam umgeht.
- Beispiel Markov-Algorithmen:
  - Können durchaus effektiv sein.
  - Sind erkennbar wenig effizient.
- Vgl. VL Komplexitätstheorie:
  - Nicht für alle Probleme können effiziente Algorithmen gefunden werden.
  - Oftmals muss ein akzeptabler Trade-Off zwischen Rechenzeit- und Speicherplatzbedarf gefunden werden.

Begriffsbildung: sequentiell vs. parallel/nebenläufig

- Ein *Algorithmus* heißt
  - sequentiell, wenn immer nur ein *Schritt* nach dem anderen ausgeführt wird.
  - parallel oder nebenläufig („potentiell parallel"), wenn gleichzeitig mehrere *Schritte* ausgeführt werden (können).
- Man unterscheidet:
  - Echte Nebenläufigkeit, z.B. bei Mehrkern-CPUs, mehreren CPUs auf dem Mainboard oder verteilten Systemen.
  - Scheinbare Nebenläufigkeit, z.B. durch präemptives Multitasking durch das Betriebssystem auf einer Einkern-CPU-Maschine.

Programme

- Programme sind Algorithmen, die in einer Maschinen- oder Programmiersprache abgefasst sind.
- Die Sprache legt die Ausdrucksmittel fest, insbesondere
  - elementare Operationen
  - Kontrollstrukturen
- Einteilung anhand von Programmierparadigmen:
  - Funktionale Programmierung Ⅰ
  - Logische Programmierung Ⅱ
  - Imperative Programmierung Ⅲ
  - Objektorientierte Programmierung Ⅳ

Ⅰ - Berechnung von Funktionswerten
- Unveränderliche Werte durch Operationen und Funktionen verknüpft

Ⅱ - Beschreibt Beziehungen zwischen Werten
↳ Auflösen via Anfragen

Ⅲ - Beschreiben Abläufe
- Nutzt veränderbare Variablen

Ⅳ - Variante der imperativen Programmierung:
-> Unterschiede bzgl. Programmorganisation
- Abbilder v. Objekten der realen Welt und deren Kommunikation durch Senden & Empfangen v. Nachrichten

Begriffsbildung: Compiler vs. Interpreter

- Ein Algorithmus A wird in einer Programmiersprache P verfasst
- Ein Computer C führt Programme in Maschinensprache M aus
  - Je nach Hersteller und Prozessorgeneration sind Maschinensprachen nicht zueinander kompatibel (z.B. x86- vs. ARM-CPUs, 32- vs. 64-bit CPUs).
  - Programme in M können deshalb i.A. nicht nach Belieben auf verschiedenen Computern C_x ausgeführt werden.
- Ein Compiler ist ein auf C in M vorliegendes Programm, das A von P nach M übersetzt. (Allgemein: Translator von P_1 nach P_2) M kann *anschließend* auf C ausgeführt werden.
- Ein Cross-Compiler läuft auf C_1 in M_1 und übersetzt A von P nach M_2. Das Resultat kann *danach* auf C_2 ausgeführt werden.
- Ein Interpreter ist ein auf C in M vorliegendes Programm, das A in P einliest und abarbeitet. M wird also zur Laufzeit erzeugt und nicht explizit gespeichert.